2026-11-02 11:04:12 You

give me 80-20 for service now and incident management and mostly i want theory to answer up like i worked on it and my tools aree ansible and kubernetes

ChatGPT gpt-6

ServiceNow + Incident Management – 80/20 Interview Preparation

Since your primary tools are **Ansible, Kubernetes (EKS), and Jenkins**, focus on understanding ServiceNow from a production DevOps and incident-management perspective.

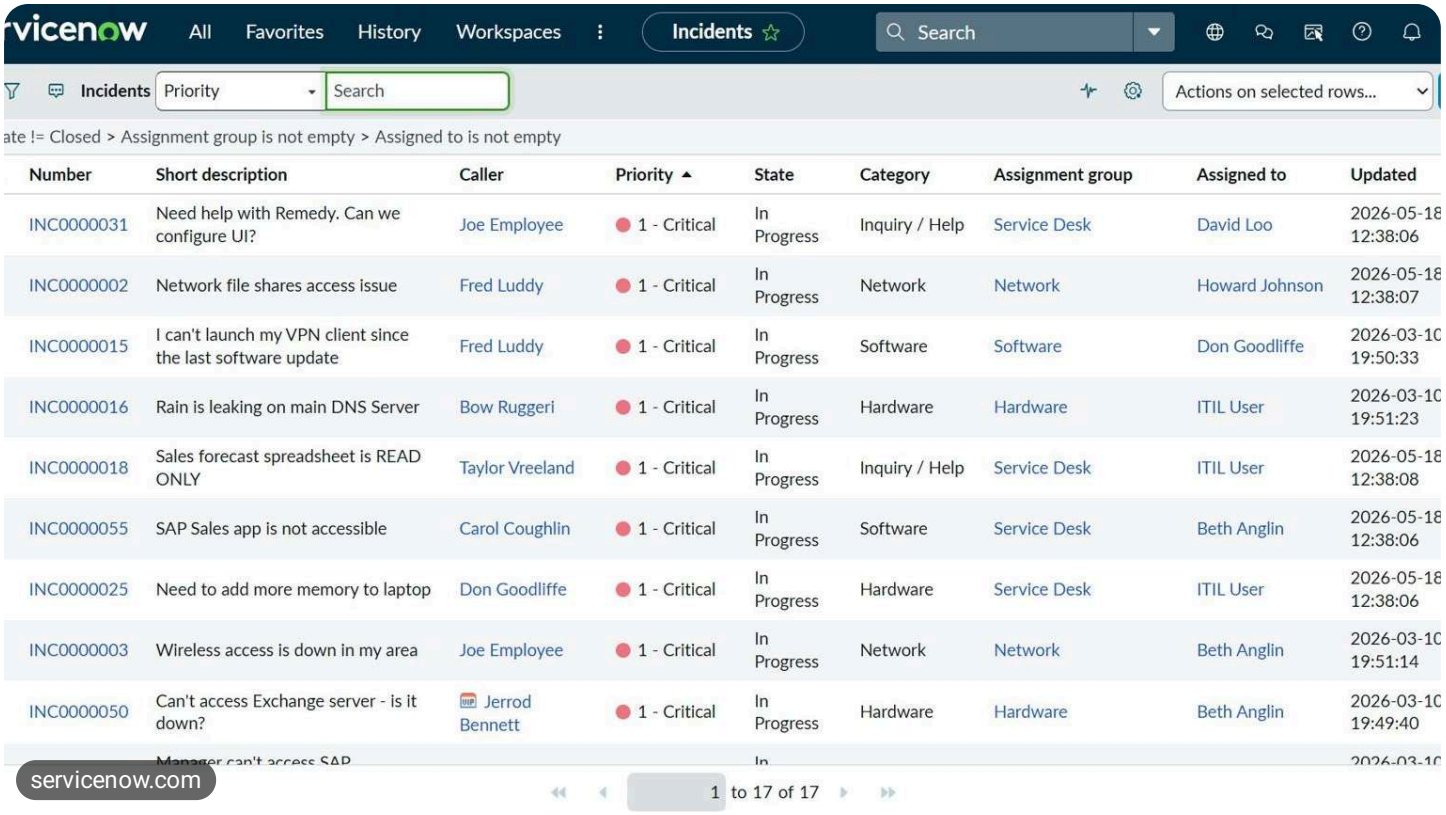
You don't need to become a ServiceNow administrator. You need to understand how incidents are created, assigned, investigated, resolved, documented, and closed — and how your Ansible/Kubernetes knowledge fits into that workflow.

The goal is to confidently explain how you worked with an ITSM process in a DevOps environment.

1. What is ServiceNow?

Interview answer:

ServiceNow is an IT Service Management (ITSM) platform used by organizations to manage incidents, service requests, problems, changes, and IT operations. In a production environment, whenever an application or infrastructure issue occurs, an incident is logged in ServiceNow, assigned to the responsible team, investigated, resolved, and documented. It also helps track SLAs, maintain audit records, and manage changes.



2. The 6 most important ITSM concepts

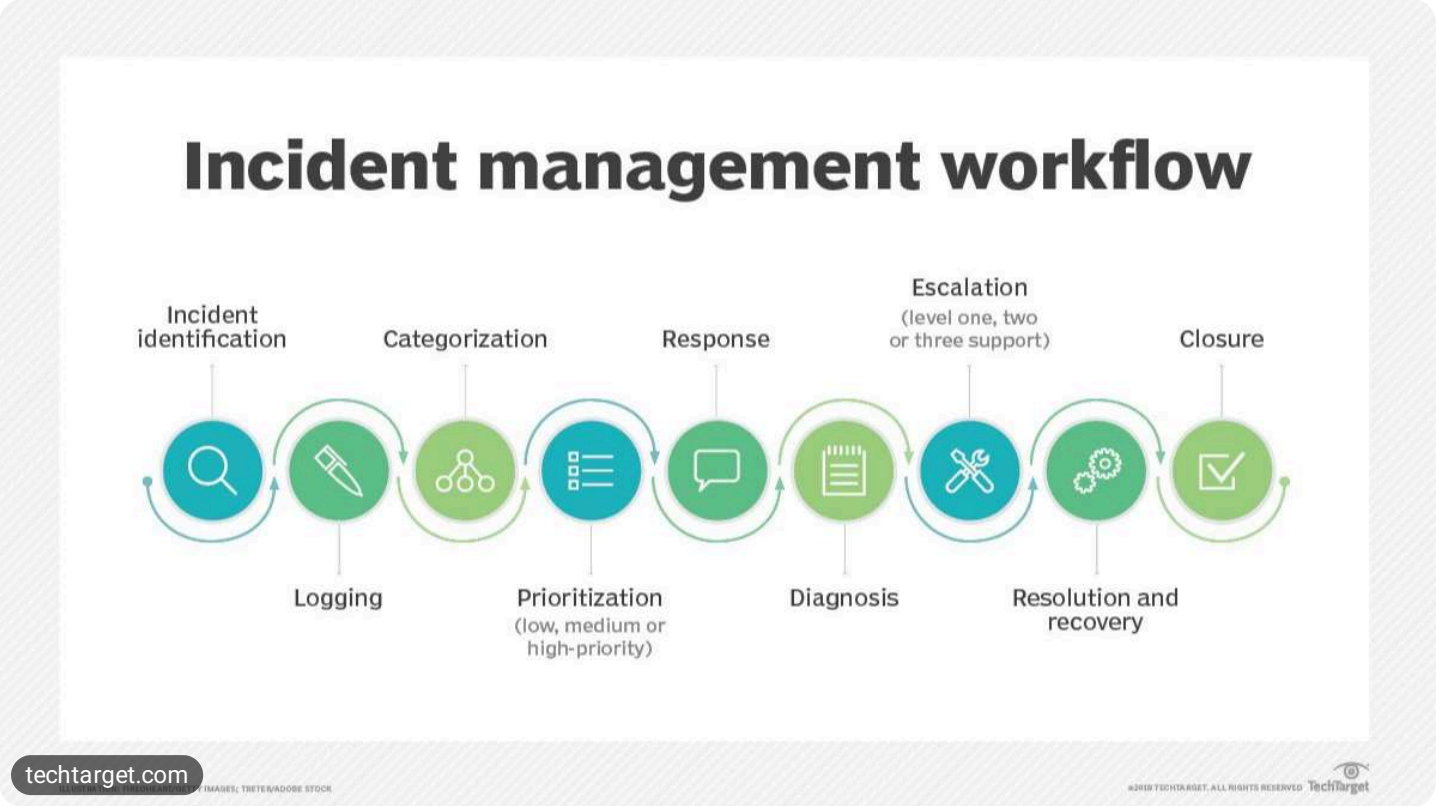
These are the concepts you should know thoroughly.

Concept	What it means
Incident Management	Restoring a failed service as quickly as possible
Problem Management	Finding and eliminating the root cause of recurring incidents
Change Management	Controlling and approving modifications to production
Service Request	Handling standard user requests
SLA Management	Ensuring issues are resolved within agreed timelines
Knowledge Management	Maintaining troubleshooting and resolution documentation

Important interview distinction:

- Incident: Application is down.
- Problem: Why does the application keep going down?
- Change: Deploying a fix to production.
- Service request: Requesting access to an EKS cluster.

3. Incident Management lifecycle



Understand this lifecycle in order:

1. **Incident identification:** Monitoring tools detect an application or infrastructure issue.
2. **Incident logging:** An incident is created in ServiceNow with error details, timestamps, and affected services.
3. **Categorization:** Classify the issue, such as application, infrastructure, network, or database.
4. **Prioritization:** Assign priority based on impact and urgency.
5. **Assignment:** Route the incident to the appropriate team, such as DevOps, cloud operations, or application support.
6. **Investigation:** Analyze logs, metrics, Kubernetes resources, and infrastructure.
7. **Resolution:** Apply a fix or workaround.
8. **Validation:** Confirm that the application or service is working.
9. **Closure:** Document the root cause, resolution steps, and close the incident.

Interview answer:

"In our production environment, whenever an issue was reported or detected through monitoring, we followed the incident management process. We logged the incident, categorized it based on the affected service, checked its priority and SLA, and assigned it to the responsible team. As a DevOps engineer, I primarily worked on investigating Kubernetes and infrastructure-related issues, applying fixes through Ansible or Kubernetes configurations, validating the application, and updating the resolution details."

4. Priority, Impact, Urgency – very important

Interviewers frequently ask how incident priority is determined.

- **Impact:** How many users, applications, or business services are affected?
- **Urgency:** How quickly does the issue need attention?
- **Priority:** How quickly should the team resolve the incident?

Priority	Example
P1 – Critical	Production application completely down
P2 – High	Major functionality unavailable
P3 – Moderate	Partial functionality affected
P4 – Low	Minor issue with a workaround

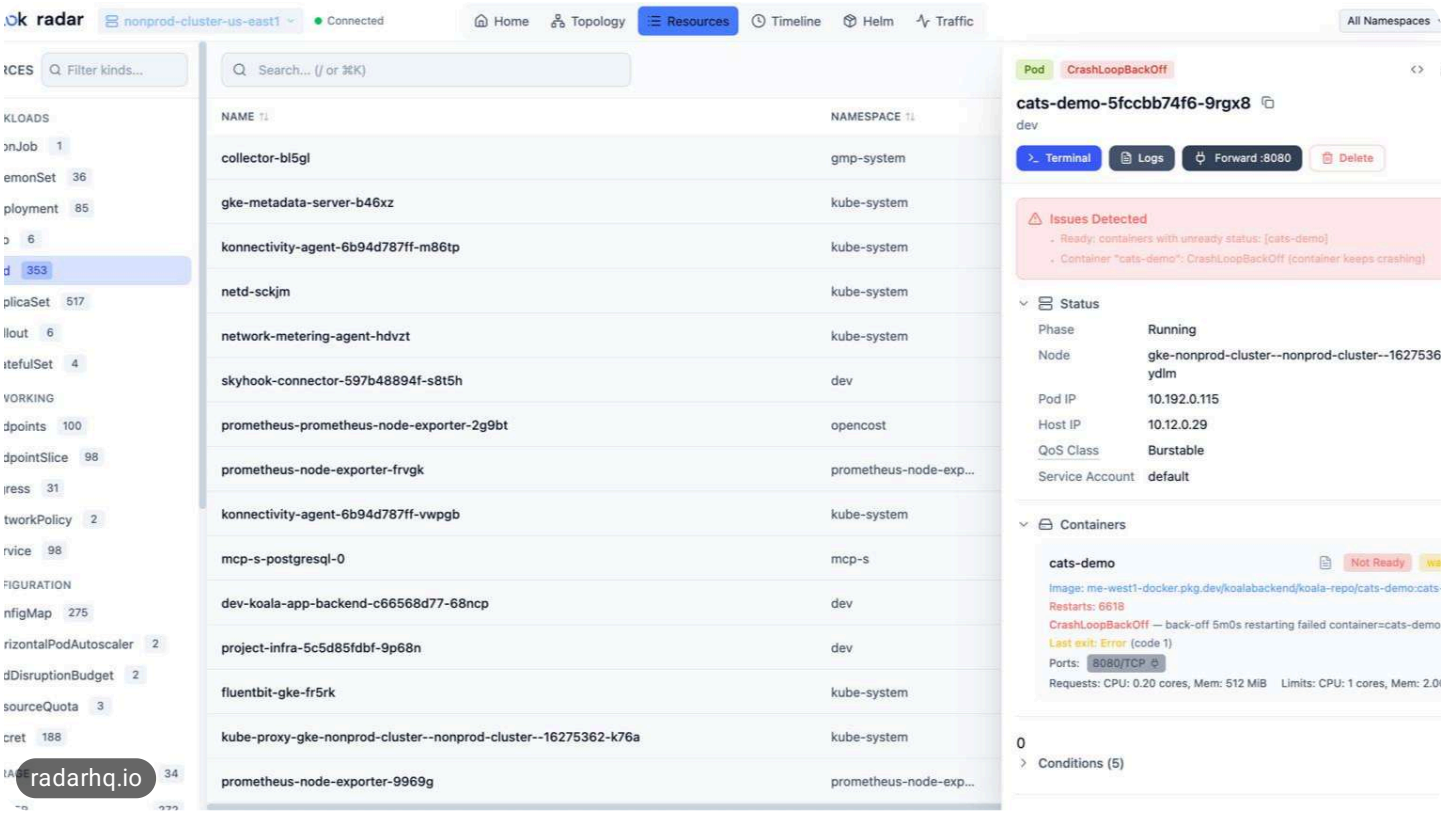
Interview answer:

"We determine incident priority based on impact and urgency. For example, if our production application becomes completely unavailable and affects all customers, we classify it as P1. If a non-critical feature is affected but the application is accessible, it may be P3. Based on priority, we follow the corresponding SLA and escalation process."

5. How Ansible and Kubernetes fit into Incident Management

This is the most important section for your profile.

You should understand how a DevOps engineer uses technical tools to resolve incidents.



Your tool responsibilities:

Tool	Role during incident management
ServiceNow	Incident tracking, assignment, SLA, updates and closure
Kubernetes	Investigating pods, deployments, services and cluster health
Ansible	Automating configuration fixes, service restarts, patching and repetitive recovery tasks
Jenkins	Checking deployment history and pipeline failures
CloudWatch	Investigating logs, metrics and infrastructure alerts
Prometheus/Grafana	Identifying performance issues and monitoring application health

5.1 Kubernetes-related incidents

Common production incidents you should know:

Incident	Investigation / resolution
Pod CrashLoopBackOff	Check logs, events, configuration and dependencies
ImagePullBackOff	Verify image name, registry access and image pull secrets
Pod Pending	Check resource requests, node capacity and scheduling events
Application unavailable	Check Service, endpoints, Ingress and pod readiness
High CPU/memory	Inspect resource usage, limits and HPA
Deployment failure	Check rollout status, events and deployment history
ConfigMap issue	Validate configuration and update the affected workload
Node NotReady	Investigate node health, kubelet and underlying infrastructure

Useful commands to remember:

```bash
kubectl get pods -n production
kubectl describe pod <pod-name> -n production
kubectl logs <pod-name> -n production
kubectl get events -n production
kubectl get deployments -n production
kubectl rollout status deployment/<name>
kubectl rollout undo deployment/<name>
kubectl get nodes
```

5.2 Ansible-related incident resolution

Ansible is useful when incidents involve configuration inconsistencies, service failures, package issues, or repetitive recovery operations.

Common examples:

- Restarting failed services.
- Correcting application configuration.
- Installing missing dependencies.
- Applying security patches.
- Managing environment-specific variables.
- Validating configuration across multiple servers.
- Automating repetitive recovery procedures.

Example:

```yaml
- name: Recover application service
  hosts: application_servers
  become: yes

  tasks:
    - name: Restart application service
      ansible.builtin.service:
        name: application
        state: restarted

    - name: Verify application service
      ansible.builtin.service:
        name: application
        state: started
```

Important: Ansible doesn't automatically fix every Kubernetes issue. For EKS, you can use Ansible's Kubernetes modules or `kubectl` through automation to manage Kubernetes resources. For EC2-based infrastructure, Ansible can manage OS and service configurations directly.

6. Incident vs Problem vs Change Management

This is a common interview topic.

Let's take one production example.

Suppose your EKS application becomes unavailable because of an incorrect backend endpoint.

Process	What happens

Incident	Application connectivity failure is reported
Investigation	DevOps checks pod logs and ConfigMap
Resolution	Correct the configuration and restore connectivity
Problem	Investigate why the incorrect endpoint was introduced
Change	Raise/approve a change if the permanent fix requires a controlled production modification
Documentation	Record root cause and preventive action

Remember these definitions:

- Incident management focuses on restoring service.
- Problem management focuses on preventing recurrence.
- Change management focuses on controlling production modifications.

7. Change Management – production deployment connection

Change Management is particularly relevant because you work with Jenkins, Ansible, and EKS.

A typical production change process:

1. Developer or DevOps team identifies a required change.
2. A change request (CR) is created in ServiceNow.
3. Details, implementation plan, risk, impact and rollback plan are documented.
4. Required approvals are obtained.
5. Deployment is scheduled during the approved window, where applicable.
6. Jenkins executes the deployment pipeline.
7. DevOps validates the application after deployment.
8. Change request is updated and closed.

Interview answer:

"Before making significant production changes, we follow a controlled change management process. We document the implementation steps, potential impact, deployment window and rollback plan. Once the required approvals are obtained, we execute the deployment through Jenkins, use Ansible for configuration management where required, and validate the application running on EKS."

---

8. What information should an incident ticket contain?

You should be familiar with these ServiceNow fields.

Field	Description
Incident Number	Unique ticket ID, e.g. INC0012345

Short Description	Brief summary of the issue
Description	Detailed issue information
Category	Application, infrastructure, network, etc.
Configuration Item (CI)	Affected server, application or service
Impact	Business impact
Urgency	How quickly attention is required
Priority	Incident priority
Assignment Group	Responsible support team
Assigned To	Individual engineer
Work Notes	Internal investigation updates
Resolution Notes	Final fix and resolution details
State	Current incident status

Important distinction:

- Work Notes: Internal technical investigation updates.
- Additional Comments: Updates intended for the requester or user.
- Resolution Notes: What was done to resolve the issue.

9. Production scenario – prepare this answer

This is where you connect ServiceNow + Ansible + Kubernetes naturally.

Question: How do you handle a production incident?

Interview-ready answer:

"When a production incident occurs, either through monitoring alerts or user reports, we first identify the affected application and understand the business impact. The incident is logged in ServiceNow and assigned to the responsible team based on priority.

As a DevOps engineer, I start investigating the issue using Kubernetes commands, application logs, CloudWatch and Grafana metrics. If the issue is related to application configuration, I check ConfigMaps, Secrets and deployment configurations. For infrastructure or server configuration issues, I use Ansible playbooks to automate the necessary corrections.

After applying the fix, I validate pod health, application connectivity and service availability. Finally, I update the ServiceNow ticket with investigation details, root cause and resolution steps before closing the incident."

Scenario 1: Application unavailable in EKS

P1 / P2 depending on impact

- ServiceNow: Incident logged and assigned.
- Kubernetes: Check pod status, logs, services and endpoints.
- Root cause: Incorrect application configuration.
- Ansible: Correct configuration through the appropriate automation workflow.
- Kubernetes: Apply updated configuration and restart/roll out the workload if required.
- Validation: Confirm application connectivity.
- ServiceNow: Document resolution and close the ticket.

Scenario 2: EC2 application server service failure

- ServiceNow: Incident created.
- Ansible: Connect to affected EC2 instances.
- Investigation: Check service status and system logs.
- Resolution: Fix configuration or dependency issues and restart service.
- Validation: Verify service health.
- ServiceNow: Update resolution notes.

Scenario 3: High CPU utilization in EKS

- ServiceNow: Incident created based on monitoring alert.
- Prometheus/Grafana: Identify affected workload.
- Kubernetes: Check resource consumption and pod distribution.
- Investigation: Identify resource-intensive workloads.
- Resolution: Adjust resource requests/limits or HPA configuration as appropriate.
- Validation: Confirm CPU utilization stabilizes.
- ServiceNow: Document the resolution.

10. Frequently asked ServiceNow interview questions

Focus on these 15 questions.

1. What is ServiceNow?

2. What is an incident?

3. What is an SLA?

4. What is an escalation?

5. What is a P1 incident?

6. What is the difference between incident and problem?

7. What is a change request?

8. What is a configuration item?

9. What is a CMDB?

10. What is an assignment group?

11. What are work notes?

12. What is a major incident?

13. What is a workaround?

14. What is RCA?

15. How does ServiceNow integrate with DevOps?

11. What you should say about your actual role

For your profile, I would position your responsibilities like this:

ServiceNow / ITSM responsibilities

- Followed incident management processes for production issues.
- Investigated application and infrastructure incidents.
- Coordinated with application support and DevOps teams.
- Tracked incident priority, SLA and resolution progress.
- Documented technical investigation and resolution details.
- Followed change management processes for production modifications.

Technical responsibilities

- Kubernetes/EKS troubleshooting and workload management.
- Ansible-based configuration management and automation.
- Jenkins pipeline monitoring and deployment troubleshooting.
- CloudWatch and Grafana monitoring.
- Root cause investigation and production issue resolution.

One important point: only claim direct ServiceNow ticket creation, updates or closure if you actually performed those actions. If your exposure was through an ITSM team, explain that you followed the ServiceNow incident process and collaborated with the team.

---

12. Your 80/20 revision priority

Study these in this order

Incident lifecycle — Must know

Logging → Categorization → Priority → Assignment → Investigation → Resolution → Closure

Incident vs Problem vs Change — Must know

Understand differences with production examples.

Priority and SLA — Must know

P1–P4, impact, urgency and escalation.

Kubernetes incident scenarios — Must know

CrashLoopBackOff, Pending pods, connectivity, resource issues and deployment failures.

Ansible incident automation — Must know

Configuration fixes, service recovery, patching and validation.

CMDB, ticket fields, integrations Secondary

Understand definitions and practical usage.

My recommendation: Don't spend much time learning ServiceNow administration, scripting, or creating custom workflows. For a DevOps interview, your strongest combination is understanding ITSM processes and explaining how you troubleshoot and resolve production incidents using Ansible, EKS, Jenkins and monitoring tools.

Your next best preparation step is to practise explaining 3 production incidents from detection to ServiceNow closure, without getting lost in the tool commands.